МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №9

"Профілювання, оптимізація та паралельні обчислення"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Свелеба С. А.

Львів-2026

Мета роботи: Варіант 1. Система обліку студентів. Мета роботи: освоїти інструменти профілювання продуктивності Python-програм (cProfile, tracemalloc, timeit), дослідити ефективність векторних обчислень на базі NumPy у порівнянні з чистою реалізацією алгоритму Велфорда (Welford algorithm), вивчити підходи до паралельної обробки даних (ThreadPoolExecutor, ProcessPoolExecutor) та реалізувати потокобезпечне LRU-кешування зі скиданням кешу за знімками стану.

Теоретичні відомості:

Профілювання дозволяє виявити 'вузькі місця' у виконанні програми за часом (CPU-bound) та використанням пам'яті (Memory-bound). Для CPU-інтенсивних завдань у CPython діє Global Interpreter Lock (GIL), тому справжній паралелізм досягається через процеси (multiprocessing / ProcessPoolExecutor) або винесення обчислень у скомпільовані C-розширення (NumPy). Для операцій вводу-виводу (I/O-bound) ефективним є використання пулу потоків (ThreadPoolExecutor). Кешування проміжних обчислень (LRU) суттєво зменшує навантаження, проте потребує суворого узгодження та інвалідації при оновленні базових даних.

Виконання роботи

1. Варіант 1: постановка задачі, предметна область та розрахунок моментів Велфорда, NumPy-аналітики та кешування

```python
"""Comparable statistics over detached Student snapshots.

Group keys use strip/casefold; tied grades retain input order. Population
standard deviation uses ddof=0. Public results contain no mutable entities.
"""

from __future__ import annotations

import math
from collections import OrderedDict
from collections.abc import Iterable
from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor
from dataclasses import dataclass, replace
from multiprocessing import get_context
from threading import Lock

from student_manager.models import Student


@dataclass(frozen=True, slots=True)
class StudentSnapshot:
    student_id: int
    first_name: str
    last_name: str
    group: str
    average_grade: float


@dataclass(frozen=True, slots=True)
class Statistics:
    count: int
    average: float | None
    minimum: float | None
    maximum: float | None
    population_stddev: float | None
```

```
    group_averages: tuple[tuple[str, float], ...]
    student_ids: tuple[int, ...]
    ranking_ids: tuple[int, ...]


def _text(value: str) -> str:
    if not isinstance(value, str):
        raise TypeError("Text fields must be strings")
    if not value.strip():
        raise ValueError("Text fields cannot be empty")
    return value.strip()


def _grade(value: float) -> float:
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise TypeError("Grade must be numeric, not bool")
    if not math.isfinite(value) or not 0 <= value <= 100:
        raise ValueError("Grade must be finite and between 0 and 100")
    return float(value)


def _student_id(value: int | None) -> int:
    if isinstance(value, bool) or not isinstance(value, int):
        raise TypeError("Student ID must be an integer")
    if value <= 0:
        raise ValueError("Student ID must be positive")
    return value


def snapshot(
    students: Iterable[Student | StudentSnapshot],
) -> tuple[StudentSnapshot, ...]:
    """Detach and validate inputs; IDs are mandatory and unique within a dataset."""
    rows: list[StudentSnapshot] = []
    seen: set[int] = set()
```

```
    for student in students:
        if not isinstance(student, (Student, StudentSnapshot)):
            raise TypeError("Expected Student or StudentSnapshot")
        identifier = _student_id(student.student_id)
        if identifier in seen:
            raise ValueError(f"Duplicate student ID: {identifier}")
        seen.add(identifier)
        rows.append(
            StudentSnapshot(
                identifier,
                _text(student.first_name),
                _text(student.last_name),
                _text(student.group).casefold(),
                _grade(student.average_grade),
            )
        )
    return tuple(rows)


def search_group(
    students: Iterable[Student | StudentSnapshot], group: str
) -> tuple[StudentSnapshot, ...]:
    """Return detached matches in input order with the same group normalization."""
    key = _text(group).casefold()
    return tuple(row for row in snapshot(students) if row.group == key)


def _summarize(rows: tuple[StudentSnapshot, ...]) -> Statistics:
    if not rows:
        return Statistics(0, None, None, None, None, (), (), ())
    count = 0
    total = 0.0
    minimum = math.inf
    maximum = -math.inf
    groups: dict[str, tuple[int, float]] = {}
```

```
    for row in rows:
        value = row.average_grade
        count += 1
        total += value
        minimum = min(minimum, value)
        maximum = max(maximum, value)
        size, subtotal = groups.get(row.group, (0, 0.0))
        groups[row.group] = size + 1, subtotal + value
    average = total / count
    squared = 0.0
    for row in rows:
        squared += (row.average_grade - average) ** 2
    return Statistics(
        count,
        average,
        minimum,
        maximum,
        math.sqrt(squared / count),
        tuple((key, subtotal / size) for key, (size, subtotal) in groups.items()),
        tuple(row.student_id for row in rows),
        tuple(
            row.student_id for row in sorted(rows, key=lambda row: -row.average_grade)
        ),
    )


def _select(
    students: Iterable[Student | StudentSnapshot], group: str | None
) -> tuple[StudentSnapshot, ...]:
    rows = snapshot(students)
    if group is None:
        return rows
    key = _text(group).casefold()
    return tuple(row for row in rows if row.group == key)
```

```
def statistics_python(
    students: Iterable[Student | StudentSnapshot], *, group: str | None = None
) -> Statistics:
    """Explicit-loop baseline, including snapshot conversion and global ranking."""
    return _summarize(_select(students, group))


def statistics_numpy(
    students: Iterable[Student | StudentSnapshot], *, group: str | None = None
) -> Statistics:
    """Vectorized moments, group reductions and stable ranking, conversion included."""
    import numpy as np

    rows = _select(students, group)
    if not rows:
        return _summarize(rows)
    grades = np.asarray([row.average_grade for row in rows], dtype=np.float64)
    group_names = tuple(dict.fromkeys(row.group for row in rows))
    codes_by_name = {name: code for code, name in enumerate(group_names)}
    codes = np.asarray([codes_by_name[row.group] for row in rows], dtype=np.intp)
    counts = np.bincount(codes)
    totals = np.bincount(codes, weights=grades)
    return Statistics(
        len(rows),
        float(grades.mean()),
        float(np.min(grades)),
        float(np.max(grades)),
        float(grades.std(ddof=0)),
        tuple(
            (name, float(totals[code] / counts[code]))
            for code, name in enumerate(group_names)
        ),
        tuple(row.student_id for row in rows),
        tuple(
```

```
            rows[int(index)].student_id for index in np.argsort(-grades, kind="stable")
        ),
    )


@dataclass(frozen=True, slots=True)
class _Moments:
    count: int
    average: float
    m2: float
    minimum: float
    maximum: float
    groups: tuple[tuple[str, int, float], ...]


def _partial_moments(rows: tuple[StudentSnapshot, ...]) -> _Moments:
    # центровані моменти не гублять малу дисперсію біля великих оцінок
    total = 0.0
    groups: dict[str, tuple[int, float]] = {}
    minimum = math.inf
    maximum = -math.inf
    for row in rows:
        total += row.average_grade
        minimum = min(minimum, row.average_grade)
        maximum = max(maximum, row.average_grade)
        size, subtotal = groups.get(row.group, (0, 0.0))
        groups[row.group] = size + 1, subtotal + row.average_grade
    average = total / len(rows)
    m2 = 0.0
    for row in rows:
        m2 += (row.average_grade - average) ** 2
    return _Moments(
        len(rows),
        average,
        m2,
```

2. Автоматизоване тестування продуктивності, експериментів та профілювання

```
PS C:\Users\ponti> pytest tests/test_performance.py tests/test_lab9_experiments.py -v
tests/test_performance.py::test_welford_empty_and_single_element PASSED
tests/test_performance.py::test_welford_accuracy_against_sample PASSED
tests/test_performance.py::test_numpy_computation_matches_welford PASSED
tests/test_performance.py::test_thread_pool_and_process_pool_execution PASSED
tests/test_performance.py::test_lru_cache_hit_and_invalidation PASSED
tests/test_lab9_experiments.py::test_quick_cli_runs_same_real_benchmarks PASSED
tests/test_lab9_experiments.py::test_real_profile_and_tracemalloc_artifacts PASSED

======================== 78 passed in 54.71s ========================
PS C:\Users\ponti> _
```

3. Відповіді на контрольні запитання

Питання: Що таке concurrency?

Відповідь: Concurrency (конкурентність) - це властивість системи керувати виконанням кількох задач шляхом перемикання між ними або чергування їхніх кроків. Вона дозволяє програмі структурувати роботу над багатьма потоками керування навіть на одному процесорному ядрі.

Питання: Що таке parallelism?

Відповідь: Parallelism (паралелізм) - це фізично одночасне виконання кількох обчислювальних операцій або потоків команд на різних ядрах чи процесорах. Він спрямований на скорочення часу виконання обчислювально важких задач за наявності апаратних ресурсів.

Питання: Чим concurrency відрізняється від parallelism?

Відповідь: Concurrency полягає в архітектурі програми та керуванні багатьма задачами одночасно без обов'язкового фізичного збігу в часі. Parallelism передбачає фізично одночасне виконання обчислень на декількох апаратних ядрах процесора.

Питання: Що таке process?

Відповідь: Process (процес) - це незалежний екземпляр програми під керуванням операційної системи, що має власну ізольовану пам'ять та ресурси. У CPython кожен процес володіє власним інтерпретатором та окремим екземпляром GIL.

Питання: Що таке thread?

Відповідь: Thread (потік) - це найменша одиниця виконання всередині процесу, що поділяє спільний адресний простір пам'яті з іншими потоками цього ж процесу. Потоки легші за процеси при створенні, проте вимагають синхронізації доступу до спільних змінних.

Питання: Чим process відрізняється від thread?

Відповідь: Процеси ізольовані за пам'яттю та вимагають міжпроцесної взаємодії (IPC), тоді як потоки одного процесу мають прямий доступ до спільної пам'яті. Перемикання контексту та створення потоків є менш витратними порівняно з повноцінними процесами.

Питання: Що таке GIL?

Відповідь: GIL (Global Interpreter Lock) - це глобальний м'ютекс у стандартній реалізації CPython, який дозволяє лише одному системному потоку одночасно виконувати байткод Python у межах процесу. Він спрощує взаємодію з C-бібліотеками та гарантує коректність внутрішнього керування пам'яттю.

Питання: Яку роль GIL відіграє у CPython?

Відповідь: GIL захищає внутрішні структури CPython та механізм підрахунку посилань (reference counting) від пошкодження при одночасному доступі потоків. Водночас це обмежує паралельне виконання чистих CPU-bound операцій Python одним ядром.

Питання: Що таке CPU-bound задача?

Відповідь: CPU-bound задача - це задача, швидкість розв'язання якої обмежена продуктивністю та тактовою частотою процесора, а не швидкістю підсистеми введення-виведення. Під час її виконання процесор безперервно завантажений інтенсивними обчисленнями.

Питання: Наведіть приклад CPU-bound задачі.

Відповідь: Прикладами CPU-bound задач є статистичні розрахунки на великих масивах, рендеринг графіки, обчислення криптографічних гешів або навчання моделей машинного навчання.

Питання: Що таке I/O-bound задача?

Відповідь: I/O-bound задача - це задача, загальний час виконання якої визначається очікуванням операцій введення-виведення, а не обчисленнями на процесорі. Більшу частину часу програма перебуває в очікуванні відповіді від диску, мережі чи бази даних.

Питання: Наведіть приклад I/O-bound задачі.

Відповідь: Прикладами I/O-bound задач є читання файлів із диску, запити до баз даних, завантаження веб-сторінок або звернення до зовнішніх REST API.

Питання: Коли доцільно застосовувати threading?

Відповідь: Threading доцільно застосовувати для I/O-bound задач, оскільки під час очікування операцій введення-виведення блокування GIL знімається й інші потоки можуть

виконувати роботу. Також потоки ефективні для виклику зовнішніх C-бібліотек, які самостійно відпускають GIL.

Питання: Коли доцільно застосовувати multiprocessing?

Відповідь: Multiprocessing доцільно застосовувати для важких CPU-bound задач, коли необхідно задіяти кілька фізичних процесорних ядер. Кожен процес отримує власний інтерпретатор та GIL, що усуває взаємне блокування обчислювального коду.

Питання: Для чого використовується Thread?

Відповідь: Клас Thread із модуля threading використовується для створення, налаштування та керування окремим потоком виконання всередині поточного процесу Python.

Питання: Для чого використовується start()?

Відповідь: Метод start() ініціалізує та запускає потік або процес в операційній системі, викликаючи його цільову функцію в новому контексті виконання.

Питання: Для чого використовується join()?

Відповідь: Метод join() призупиняє виконання викликаючого потоку до моменту повного завершення роботи цільового потоку або процесу.

Питання: Що таке race condition?

Відповідь: Race condition (стан гонитви) - це дефект у багатопотоковій системі, коли кінцевий стан даних залежить від неконтрольованого порядку перемикання потоків при роботі зі спільним ресурсом.

Питання: Що таке critical section?

Відповідь: Critical section (критична секція) - це ділянка коду, в якій відбувається читання або модифікація спільного змінюваного ресурсу, куди не можна одночасно пускати більше одного потоку.

Питання: Для чого використовується Lock?

Відповідь: Lock (примітив блокування) використовується для синхронізації потоків, гарантуючи монопольний доступ одного потоку до критичної секції у будь-який момент часу.

Питання: Що таке synchronization?

Відповідь: Synchronization (синхронізація) - це координація виконання паралельних потоків або процесів для впорядкованого доступу до спільних ресурсів і запобігання конфліктам даних.

Питання: Чи завжди synchronization покращує performance?

Відповідь: Ні, синхронізація додає накладні витрати на перемикання контексту й очікування на примітивах блокування, що нерідко знижує швидкодію. Вона використовується виключно для забезпечення коректності та цілісності даних.

Питання: Що таке ThreadPoolExecutor?

Відповідь: ThreadPoolExecutor - це високорівневий клас із concurrent.futures, який керує пулом потоків і чергою задач, повторно використовуючи існуючі потоки.

Питання: Коли його доцільно використовувати?

Відповідь: Його доцільно використовувати для масового виконання асинхронних I/O-bound операцій, таких як завантаження файлів чи одночасні мережеві виклики.

Питання: Що таке ProcessPoolExecutor?

Відповідь: ProcessPoolExecutor - це високорівневий пул процесів із модуля concurrent.futures, призначений для автоматичного розподілу задач між окремими системними процесами Python.

Питання: Коли його доцільно використовувати?

Відповідь: Його доцільно використовувати для важких незалежних CPU-bound задач, де час самих обчислень перевищує накладні витрати на запуск процесів та обмін даними.

Питання: Для чого використовується executor.map()?

Відповідь: Метод executor.map() асинхронно застосовує задану функцію до елементів ітерованої послідовності у воркерах пулу, повертаючи результати у порядку початкових елементів.

Питання: Що таке worker?

Відповідь: Worker (робочий потік або процес) - це окрема сутність пулу, яка в нескінченному циклі отримує задачі з черги, виконує їх та повертає результат.

Питання: Чи завжди більша кількість workers дає більшу швидкість?

Відповідь: Ні, занадто велика кількість воркерів викликає надмірні накладні витрати на перемикання контексту ОС, брак апаратних ядер та конкуренцію за ресурси пам'яті.

Питання: Що таке multiprocessing overhead?

Відповідь: Multiprocessing overhead - це додаткові ресурси процесора та пам'яті, які витрачаються на створення нових процесів, серіалізацію об'єктів та міжпроцесний зв'язок (IPC).

Питання: Що таке serialization?

Відповідь: Serialization (серіалізація) - це перетворення об'єктів Python у двійковий потік байтів (зазвичай через pickle) для передачі між процесами або збереження у сховищі.

Питання: Чому multiprocessing може бути повільним для маленьких задач?

Відповідь: Для невеликих задач час створення нових процесів і серіалізації вхідних та вихідних даних через IPC набагато перевищує час самого обчислення, сповільнюючи програму.

Питання: Для чого використовується timeit?

Відповідь: Модуль timeit використовується для точного бенчмаркінгу невеликих фрагментів коду Python шляхом їхньої багаторазової ітерації з мінімізацією зовнішніх похибок.

Питання: Для чого використовується perf_counter()?

Відповідь: Функція time.perf_counter() використовується для вимірювання інтервалів часу виконання функцій або етапів програми з найвищою роздільною здатністю таймера ОС.

Питання: Навіщо виконувати декілька benchmark repeats?

Відповідь: Декілька повторів бенчмарку необхідні для нівелювання впливу системних шумів, фонової активності ОС, дій збирача сміття та отримання стабільного середнього або медіани.

Питання: Що таке profiling?

Відповідь: Profiling (профілювання) - це аналіз продуктивності програми за допомогою вимірювання часу виконання окремих функцій, частоти їхніх викликів та споживання системних ресурсів.

Питання: Для чого використовується cProfile?

Відповідь: Модуль cProfile використовується для детермінованого профілювання Python-коду з фіксацією тривалості й кількості викликів усіх функцій для пошуку вузьких місць.

Питання: Що показує ncalls?
Відповідь: Параметр ncalls у звіті cProfile показує сумарну кількість викликів зазначеної функції за весь період профілювання.

Питання: Що показує tottime?
Відповідь: Параметр tottime показує чистий час, витрачений виключно на виконання коду даної функції, без урахування часу вкладених викликів інших функцій.

Питання: Що показує cumtime?
Відповідь: Параметр cumtime показує накопичений сумарний час, витрачений на виконання цієї функції разом із усіма її вкладеними функціями та підпрограмами.

Питання: Що таке bottleneck?
Відповідь: Bottleneck (вузьке місце) - це ділянка або функція коду, яка виконується найдовше або споживає найбільше ресурсів, визначаючи граничну швидкість усієї програми.

Питання: Як знайти bottleneck?
Відповідь: Bottleneck знаходять шляхом запуску коду під профілювальником cProfile, сортуючи звіт за cumtime та tottime для виявлення найбільш навантажених функцій.

Питання: Для чого використовується tracemalloc?
Відповідь: Модуль tracemalloc використовується для відстеження алокації блоків пам'яті інтерпретатором Python, дозволяючи фіксувати поточне та пікове використання пам'яті за рядками коду.

Питання: Що таке peak memory?
Відповідь: Peak memory (пікова пам'ять) - це максимальний обсяг оперативної пам'яті, одночасно виділений програмою за весь період вимірювання.

Питання: Що таке NumPy?
Відповідь: NumPy - це оптимізована бібліотека для Python, яка надає структури багатовимірних масивів і скомпільовані математичні операції над ними, написані мовами C та Fortran.

Питання: Що таке vectorization?
Відповідь: Vectorization (векторизація) - це техніка виконання операцій над цілими масивами даних одночасно без використання повільних інтерпретованих циклів мови Python.

Питання: Чому NumPy часто швидший за Python loops?
Відповідь: NumPy швидший за цикли Python, оскільки оперує однорідними даними у суцільних блоках пам'яті, виконує цикли на рівні C та ефективно задіює векторні SIMD-інструкції процесора.

Питання: Що таке caching?
Відповідь: Caching (кешування) - це техніка оптимізації, яка зберігає результати дорогих операцій у пам'яті, дозволяючи миттєво повертати їх при повторних однакових запитах.

Питання: Для чого використовується @cache?
Відповідь: Декоратор functools.cache використовується для автоматичного мемоізування функцій без обмеження кількості збережених результатів, кешуючи повернені значення за аргументами.

Питання: Для чого використовується @lru_cache?

Відповідь: Декоратор functools.lru_cache використовується для кешування результатів із обмеженням максимального розміру пам'яті, витісняючи найменш вживані елементи за алгоритмом LRU.

Питання: Що таке cache hit?

Відповідь: Cache hit (влучання в кеш) - це ситуація, коли запитуване значення вже присутнє в кеші та повертається миттєво без повторного виконання тіла функції.

Питання: Що таке cache miss?

Відповідь: Cache miss (промах кешу) - це ситуація, коли результат для переданих аргументів відсутній у кеші, тому викликається цільова функція, а її результат зберігається в кеш.

Питання: Що таке cache invalidation?

Відповідь: Cache invalidation (інвалідація кешу) - це скидання або видалення застарілих даних із кешу при зміні вихідного стану системи для запобігання видачі неактуальних результатів.

Питання: Коли кешування може бути небезпечним?

Відповідь: Кешування небезпечне при роботі з непостійними або змінюваними даними без належної інвалідації, для функцій із побічними ефектами, а також у разі переповнення оперативної пам'яті через відсутність обмежень розміру кешу.

Питання: Як обчислюється speedup?

Відповідь: Speedup (коефіцієнт прискорення) обчислюється як відношення часу виконання базової версії до часу виконання оптимізованої версії: T_baseline / T_optimized.

Питання: Що означає speedup менше 1?

Відповідь: Speedup менше 1 означає уповільнення роботи (деградацію), тобто оптимізована версія виконалась повільніше за базову через накладні витрати або неефективність методу.

Питання: Чому threaded CPU-bound version може бути повільнішою?

Відповідь: Threaded CPU-bound версія може бути повільнішою через дію GIL, що обмежує виконання одним ядром, разом із додатковими накладними витратами на перемикання контексту та керування потоками.

Питання: Чому ProcessPoolExecutor може прискорити CPU-bound код?

Відповідь: ProcessPoolExecutor може прискорити CPU-bound код, оскільки кожен процес запускається зі своїм незалежним екземпляром інтерпретатора та GIL, розподіляючи обчислення по фізичних ядрах.

Питання: Чому необхідно перевіряти правильність optimized implementation?

Відповідь: Перевірка правильності необхідна тому, що оптимізація чи паралелізація можуть внести помилки округлення, порушити черговість операцій або спричинити пошкодження стану через race conditions.

Питання: Чому оптимізацію потрібно починати з profiling?

Відповідь: Оптимізацію слід починати з профілювання, щоб виявити справжній bottleneck програми та не витрачати ресурси на оптимізацію несуттєвих ділянок коду, які не впливають на загальний час.

Висновок:

У ході роботи реалізовано високопродуктивний модуль статистичної аналітики з алгоритмом Велфорда та оптимізацією через NumPy, паралельну обробку масивів через ThreadPool та ProcessPool, а також потокобезпечний сервіс кешування з підтримкою

інвалідації. Досліджено профілі викликів та споживання оперативної пам'яті за допомогою cProfile і tracemalloc. Усі 78 тестів продуктивності успішно пройдено.